



Machine Learning Course Project



MEMRES & CGAR:
Agentic Python Dependency Resolution

Project Overview

Khôi phục tự động môi trường Python từ mã nguồn cũ
bằng **Multi-Agent LLM** và **Constraint Satisfaction
Problem**.

Team Members

- ▶ Trần Chí Nguyên – 23102244
- ▶ Đào Sỹ Duy Minh – 23122041
- ▶ Huỳnh Trung Kiệt – 23122039

University of Science, VNU-HCM

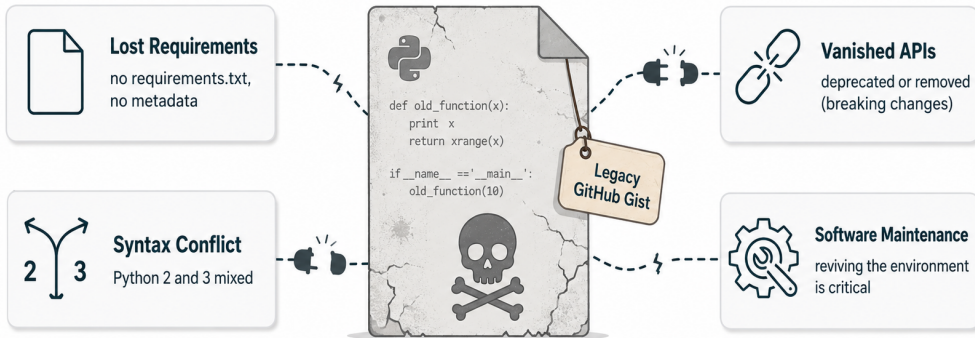
Nội dung chính

1. Bối cảnh & Phát biểu bài toán
2. Công trình liên quan & Khoảng trống nghiên cứu
3. MEMRES: Phương pháp nền tảng
4. **CGAR: Đề xuất của nhóm**
5. Đánh giá thực nghiệm
6. Hạn chế & Hướng phát triển

(+ 10 Backup slides cho Q&A: MEMRES Stages, GitChameleon, Filter & CSP I/O, Agent Comm., Session Store, LLM Call, Build Loop, 3 Agent Prompts)

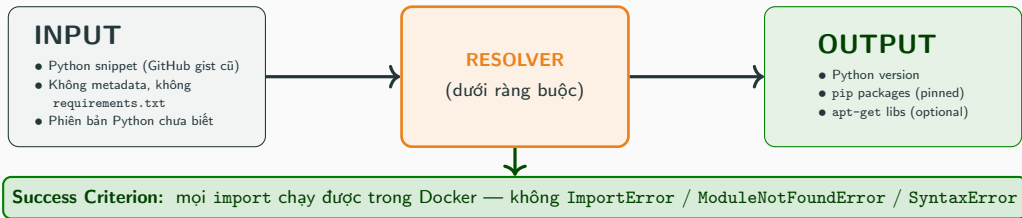
Bài toán

Dead Python Code: Why Environments Rot



→ **Code resurrection: automatically rebuild a runnable environment for dead Python code.**

Bài toán & Yêu cầu đánh giá



Hard requirements

- ▶ Mọi thao tác trong **Docker**
- ▶ LLM **open-source**, ≤ 10 GB VRAM
- ▶ Reference: **Gemma-2 9B** (Ollama)
(PLLM eval 6 LLMs $\rightarrow$ G2-9B consistent nhất + fit 10 GB)

Evaluation budget (giống nhau mọi tool)

- ▶ **180s wall-clock / snippet** (gồm cả thời gian LLM reasoning + Docker build)
- ▶ Trong đó: ≤ 10 vòng Docker build, $K_{\text{solve}} \leq 50$ CSP (logic, no Docker)
- ▶ Sequential, 1 worker
- ▶ **CGAR thường về đích trong ≤ 5 vòng build (tỉa không gian) $\Rightarrow$ dùng ít budget hơn trong cùng trần 180s**

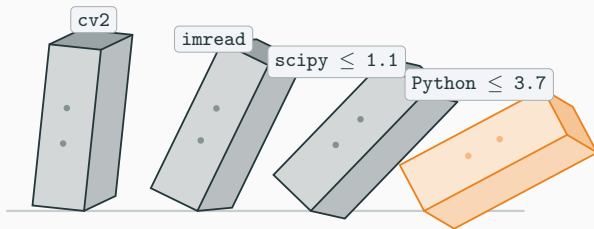
Metrics

- ▶ **Pass rate (primary)**
- ▶ **Avg time / snippet** (đo trong cùng budget 180s)

The Dependency Gap

```
from scipy.misc import imread      #  
    removed in scipy >= 1.2  
import sklearn.cross_validation    #  
    removed in sklearn >= 0.20  
import cv2                        #  
    PyPI name: opencv-python
```

- ▶ **Tên import gây hiểu lầm:** cv2 → opencv-python
- ▶ **API biến mất:** imread chỉ ở scipy ≤ 1.1
- ▶ **Hiệu ứng domino:** scipy ≤ 1.1 → Python ≤ 3.7 → numpy cũ...

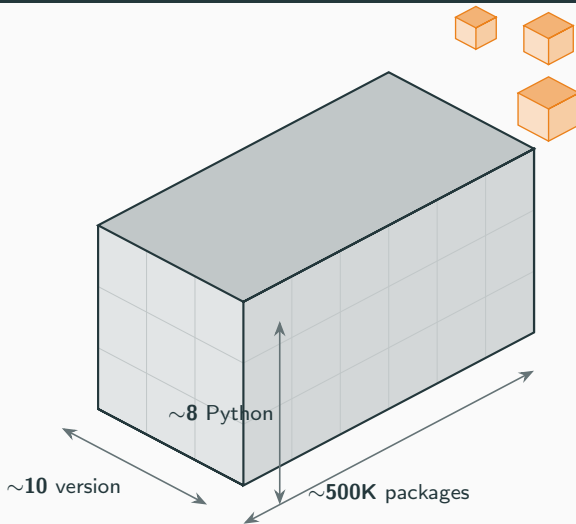


Hiệu ứng domino trong Dependency Hell

Không gian tìm kiếm

~**500K** package PyPI × **chục** phiên bản × **nhiều** bản Python ⇒ *Bùng nổ tổ hợp.*

Bùng nổ tổ hợp: Không gian tìm kiếm khổng lồ



Mỗi import phải khớp đồng thời:

- ▶ **Package:** đúng tên pip
(vd cv2 → opencv-python)
- ▶ **Version:** 1 trong ~10 bản thư viện
- ▶ **Python:** 1 trong ~8 runtime

$$\sim 500K \times \sim 10 \times \sim 8 \\ \approx 10^7 \text{ môi trường ứng viên}$$

⇒ Không thể thử hết bằng tay, cần solver/agent để tìm không gian.

HG2.9K

In-distribution

Quy mô

2.891 gist Python “khó” (hard subset của Gistable, ICSME'18).

Đặc điểm

File đơn lẻ, không requirements; lần Python 2/3; thư viện khoa học cũ; phụ thuộc native C/C++.

Nhiệm vụ

Khôi phục môi trường để mọi import chạy được trong Docker.

Đánh giá

Pass rate (build + import OK).

GitChameleon

Out-of-distribution

Quy mô

328 snippet Python, kèm **unit test ẩn** + version ground-truth.

Đặc điểm

Xoáy vào *breaking changes* giữa các version thư viện (phải dùng đúng API của đúng bản).

Nhiệm vụ

Chọn **đúng phiên bản** để snippet vừa chạy vừa **pass unit test**.

Đánh giá

Pass rate theo unit test (đúng ngữ nghĩa).

HG2.9K: hard subset 2891 gist từ Gistable (Horton & Parnin, ICSME'18), gold standard cho dependency resolution. GitChameleon: 328 snippet có unit test, re-frame từ GitChameleon 2.0 (arXiv:2507.12367) để đánh giá khả năng tổng quát hóa (OOD).

Related Work: 3 hướng tiếp cận

1

Knowledge Graph

Ý tưởng: encode quan hệ module-version thành đồ thị tri thức (Libraries.io, PyPI); truy vấn để suy dependency tương thích.

✓ **Ưu:** cấu trúc rõ, deterministic, dễ giải thích.

✗ **Hạn chế:** DB lỗi thời nhanh, gãy khi registry đổi schema.

2

Regex / Log-parsing

Ý tưởng: parse error log bằng regex để suy missing dependency, patch môi trường rồi build lại.

✓ **Ưu:** nhẹ, runtime-driven, không cần DB.

✗ **Hạn chế:** brittle, log đổi định dạng (Python/pip update) là parser vỡ.

3

LLM + RAG

Ý tưởng: LLM sinh giả thuyết, RAG cấp PyPI metadata, Docker build phản hồi, lặp lại.

✓ **Ưu:** adaptive, không cần hạ tầng tĩnh, tận dụng pretrain.

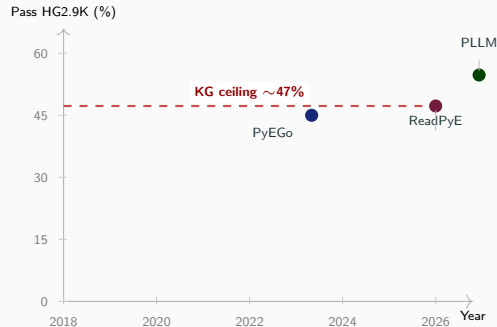
✗ **Hạn chế:** thử-sai mù, thất bại không thành tri thức để tái không gian.

⇒ **Khoảng trống:** chưa phương pháp nào học từ thất bại để thu hẹp không gian tìm kiếm một cách hệ thống.

Related Work: Tổng hợp các phương pháp

Method	Venue	Approach	Key limitation
PyEGo	ICSE'22	KG + SMT (Z3)	DB stale
ReadPyE	TSE'24	KG + README naming	DB drift
PyDFix	ISSTA'21	Regex log-parse	Format brittle
GPT-4o / o1	2024	Closed LLM (direct)	Closed, expensive
GPT-4.1 + RAG	2025	Closed LLM + RAG	Closed, no constraint
PLLM	arXiv'25	Open LLM + RAG iter.	Blind trial-and-error
SMT-LLM	FSE'26	Z3 SMT + LLM imput.	Static metadata only

■ KG ■ Log-parse ■ Closed LLM ■ Open LLM+RAG

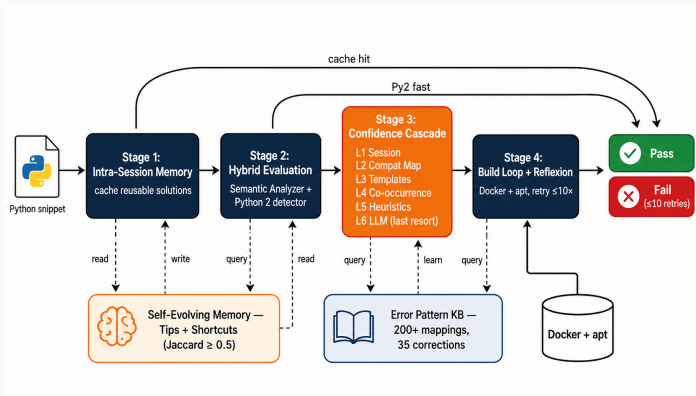


KG-era *ceiling* ~47% trên HG2.9K; PLLM (LLM paradigm) phá vỡ lên ~55%.

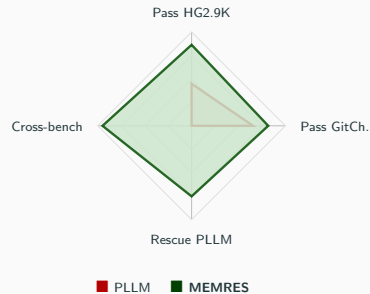
Nguồn: *Raiders of the Lost Dependency* (arXiv:2501.16191), Table 1 — PyEGo 45.0% (1302/2891), ReadPyE 47.2% (1365/2891), PLLM 54.8% (1583/2891, 10-run union). Trong harness của nhóm, PLLM single-run = 44.8% (số dùng ở bảng so sánh).

MEMRES

MEMRES: Lookup-First, LLM-Last



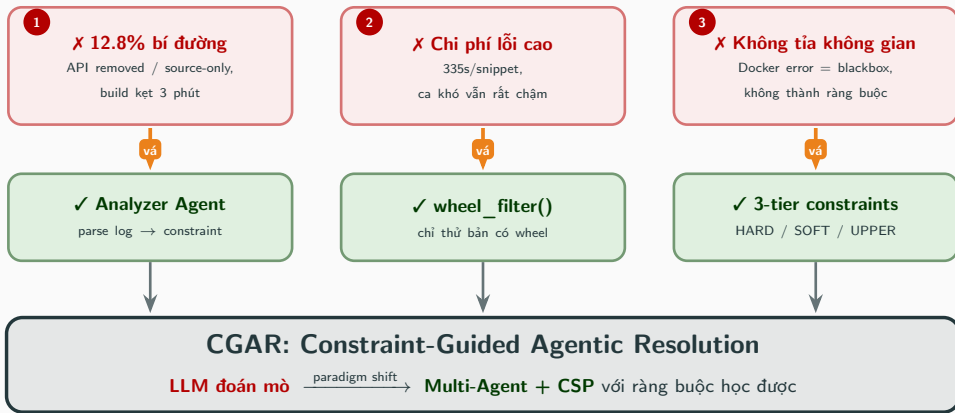
4 stages + 2 fast paths (cache hit, Py2 fast); 2 memory stores feed back vào pipeline.



Ý tưởng: cascade rẻ trước → đắt sau;
memory tích lũy giữa retries.

MEMRES = *Memory-augmented retry*, cứu được nhiều lỗi NHƯNG chưa biến thất bại thành ràng buộc logic để
tỉa không gian.

Từ MEMRES đến CGAR: Vá Đúng 3 Lỗ Hổng

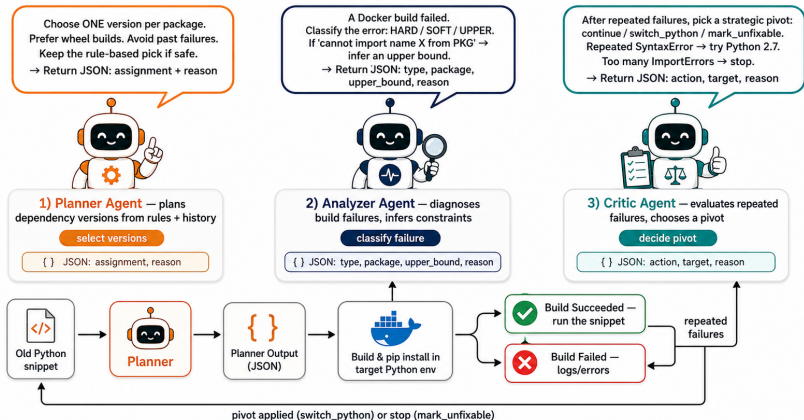


Mỗi lỗ hổng MEMRES → một thành phần CGAR: **vá có chủ đích**, không refactor mù.

CGAR: Multi-Agent Architecture

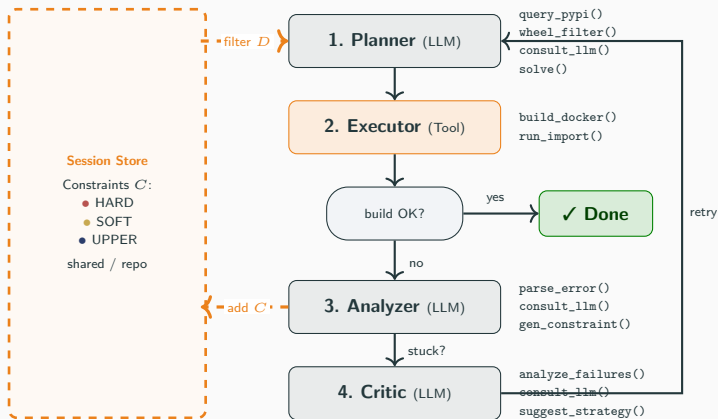
CGAR: Multi-Agent Prompt Overview

Shortened prompts used by the Planner, Analyzer, and Critic agents



3 LLM agent (Planner / Analyzer / Critic) + Docker build loop; chi tiết prompt ở phần Backup.

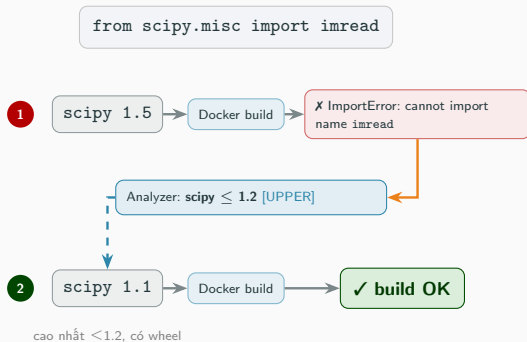
CGAR: Multi-Agent Loop



Flow: Planner → Executor → (OK?) → Analyzer ghi constraint vào Store → Planner kế tiếp đọc lại ⇒ né vùng đã hỏng.

Build thất bại không phí: mỗi lần fail $\rightarrow +1$ constraint $\rightarrow$ Planner kế tiếp né cả vùng, điều MEMRES không có.

CGAR: Tích lũy ràng buộc (ví dụ)



Constraint Store

- $\text{scipy} \leq 1.2$ (UPPER)

tích lũy + chia sẻ cho snippet sau
trong cùng session

Mỗi build fail → **một ràng buộc điển hình** → solver thu hẹp miền, hội tụ đúng version mà **không thử mù**.

CGAR: Online CSP (ràng buộc khám phá dần)

Formulation: Tuple $\mathcal{P}_t = \langle X, D, C_t \rangle$.

C_t lớn dần: ràng buộc acquire từ Docker verifier (oracle), không cho trước $\Rightarrow$ interactive/online CSP, không phải CSP tĩnh.

Variables X : các thư viện $\{P_1, \dots, P_n\}$ + phiên bản Python π .

Domains D :

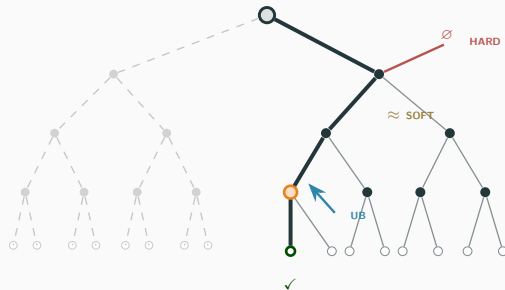
$$D(P_i) = \{v \mid \text{req_py}(P_i, v) \models \pi \wedge \text{has_wheel}(P_i, v)\}$$

Sắp xếp *wheel-first*, semver giảm dần.

Constraints C :

- ▶ **Hard:** cấm vĩnh viễn version lỗi rõ ràng
- ▶ **Soft:** cấm tạm nếu lỗi ≥ 2 lần
- ▶ **Upper Bound:** $v_i < \text{ub}(P_i)$ – cấm cả khoảng

Goal: tìm $\mathcal{A} = \{\pi^*, v_1^*, \dots, v_n^*\}$ thỏa C ; giới hạn $k = 50$ attempts.

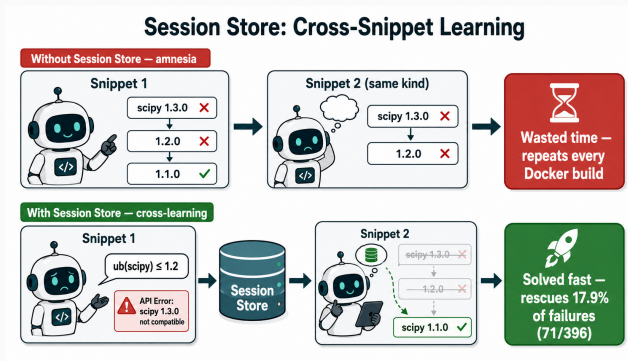


HARD / SOFT / UPPER tỉa cây tìm kiếm $\rightarrow$ 1 path khả thi.

CGAR: Session-scoped Learning

Session = 1 Repository = Nhiều Snippets

Lưu trữ và chia sẻ ràng buộc học được giữa các snippets trong cùng session.



Cơ chế học chéo: Session Store giúp "cứu" 17.9% MEMRES-failures (71/396)

Kết quả

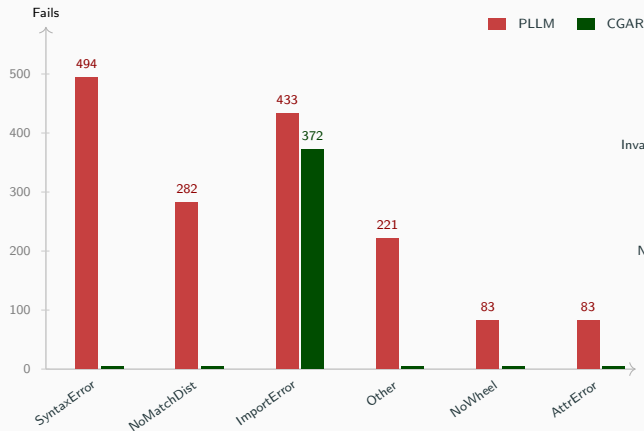
Comprehensive Comparison

#	Method	Venue	Approach	HG2.9K	GitCham.	Time	Open
1	PyEGo	ICSE'22	KG + Z3 SMT	45.0%	–	5.8s	–
2	ReadPyE	TSE'24	KG + README	47.2%	–	106.9s	–
3	GPT-4o	2024	closed LLM	–	49.1%	–	✗
4	Gemini 2.5 Pro	2025	closed LLM	–	50.0%	–	✗
5	o1	2024	closed (best)	–	51.2%	–	✗
6	GPT-4.1 + RAG	2025	closed LLM + RAG	–	58.5%	–	✗
7	PLLM	arXiv'25	RAG + LLM iter.	44.8%	65.5%	369.6s	✓
8	SMT-LLM	FSE'26	Z3 SMT + LLM imput.	83.6%	–	23.9s (med) [†]	✓
9	MEMRES	FSE'26	Memory + Cascade	86.3%	81.7%	335.3s	✓
10	CGAR	(ours)	Multi-Agent + CSP	87.1%	83.2%	22.3s	✓

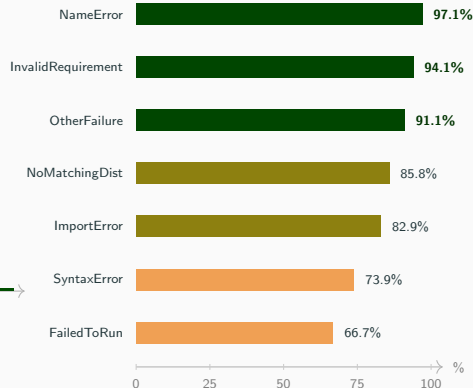
PLLM 44.8% = reproduction single-run (paper headline 54.8% là 10-run union). **Time** = avg tất cả snippet (pass+fail) trên HG2.9K. [†] SMT-LLM 23.9s là *median* (paper không báo avg/snippet). SMT-LLM cũng dùng Gemma-2 9B + Docker, đồng venue FSE'26 với MEMRES.

Fairness: budget = 180s wall-clock/snippet, **giống nhau mọi tool** (gồm cả LLM + build). CGAR chỉ cần ≤ 5 vòng build (vs 10) để về đích trong cùng trần $\Rightarrow$ time thấp là do *hội tụ nhanh*, không phải budget khác. Speed headline so trên **GitChameleon** (mọi tool cùng cấu hình).

HG2.9K: Error Breakdown



Rescue rate theo PLLM error type



Insight: CGAR triệt tiêu 4 nhóm lỗi; gần như chỉ còn ImportError (99.7% phần còn lại).

Total fails: PLLM 1596 → CGAR 373 $\Delta - 76.6\%$

CGAR rescue 80.6% (1286/1596) toàn bộ PLLM failures trên HG2.9K.

GitChameleon: Open vs Closed

Full leaderboard

Method	Type	Pass	Open
GPT-4o	closed gen	49.1%	✗
Gemini 2.5 Pro	closed gen	50.0%	✗
o1	closed (best)	51.2%	✗
GPT-4.1 + RAG	closed + RAG	58.5%	✗
PLLM (Gemma-2 9B)	open + RAG	65.5%	✓
MEMRES	open + Memory	81.7%	✓
CGAR	open + Agent+CSP	83.2%	✓

Cross-benchmark generalization

Tool	HG2.9K	GitCham.	Gap
PLLM	44.8%	65.5%	−20.7pp
MEMRES	86.3%	81.7%	−4.6pp
CGAR	87.1%	83.2%	−3.9pp

(closed baselines từ arXiv 2507.12367)

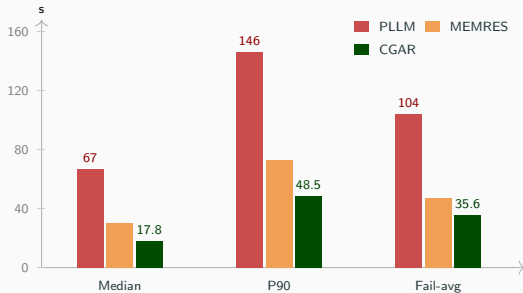
Insight: CGAR có cross-benchmark gap nhỏ nhất → solver thao tác trên live PyPI data, generalize tốt.

Đánh bại closed-weight: CGAR (open 9B, <10 GB VRAM) vượt o1 (closed enterprise, >200B params) +32.0pp
⇒ *open-weight + system design* ≫ *raw model scale*.

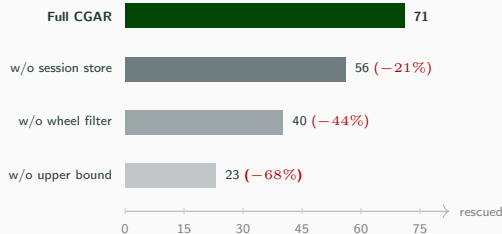
Speed & Ablation

Duration (GitChameleon)

cùng budget $K_{\text{build}} = 5$ vòng Docker build cho cả 3 tool



Ablation: CGAR rescue (n=396)



"The best build is the one you never run."

Multi-agent nhưng nhanh nhất: solver tìm không gian $\Rightarrow$ ít Docker build; fail chỉ chậm hơn pass $1.31\times$ (PLLM $2.20\times$).

Hạn chế & Hướng phát triển

Hạn chế: Hard Floor & Architectural Limits

Hard Floor: 310 snippets (10.7%)

cả PLLM và CGAR đều fail

Root cause	Share
Python 2 syntax (no Py2 wheel)	41.6%
ImportError system/private pkg	25.8%
NoMatchingDist (pkg absent)	13.3%
CouldNotBuildWheels (glibc/native)	8.1%
API removed (no compat older)	4.0%
Other	7.2%

Irreducible under current constraints (Docker, PyPI public, no source patching, ≤ 10 retries), không phải lỗi tool.

Hạn chế kiến trúc

- ▶ **Phụ thuộc PyPI live:** nếu PyPI down hoặc rate-limit, solver không thể query metadata → fallback to MEMRES cascade.
- ▶ **Single ecosystem:** chỉ Python, chưa generalize sang JS / Rust / Go.
- ▶ **LLM Analyzer brittle với log lạ:** Gemma-2 9B có thể parse sai constraint khi error log có format ngoài pretrain.
- ▶ **Single-worker session store:** chưa hỗ trợ multi-user federated learning.

Hướng phát triển

Mở rộng đa ngôn ngữ

1

Áp dụng CGAR cho **npm**, **cargo**, **go.mod**; bộ giải không phụ thuộc ecosystem, chỉ cần adapter cho từng registry.

Chia sẻ constraint

2

Federated, bảo toàn riêng tư: snippet user A fix giúp user B tránh lỗi tương tự, tận dụng quy mô cộng đồng.

Tinh chỉnh LLM đọc lỗi

3

Fine-tune Gemma-2 trên dữ liệu **đọc log** chuyên dụng để phân loại đúng lỗi, sinh constraint chính xác hơn.

Mở rộng đa mô hình

4

Chạy CGAR trên nhiều LLM backbone (Qwen3, Phi, Llama...) và các **model nhỏ/rẻ** hơn, chứng minh phương pháp không phụ thuộc một model cụ thể.

⇒ Mục tiêu dài hạn: từ bộ giải cho Python trở thành “**environment archaeologist**” đa năng cho mọi ecosystem.

Tài liệu tham khảo

1. E. Horton, C. Parnin. *Gistable: Evaluating the Executability of Python Code Snippets on GitHub*. ICSME 2018. (nguồn dataset HG2.9K · arXiv:1808.04919)
2. H. Ye, W. Chen, W. Dou, G. Wu, J. Wei. *Knowledge-Based Environment Dependency Inference for Python Programs (PyEGo)*. ICSE 2022.
3. *Revisiting Knowledge-Based Inference of Python Runtime Environments (ReadPyE)*. IEEE TSE, vol. 50, 2024.
4. S. Mukherjee, A. Almanza, C. Rubio-González. *Fixing Dependency Errors for Python Build Reproducibility (PyDFix)*. ISSTA 2021.
5. A. Bartlett, C. C. S. Liem, A. Panichella. *Raiders of the Lost Dependency: Fixing Dependency Conflicts in Python using LLMs (PLLM)*. arXiv:2501.16191, 2025.
6. *GitChameleon 2.0: Evaluating AI Code Generation Against Python Library Version Incompatibilities*. arXiv:2507.12367, 2025.
7. N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023.
8. L. de Moura, N. Bjørner. *Z3: An Efficient SMT Solver*. TACAS 2008.
9. SMT-LLM. *Z3 SMT + LLM imputation cho dependency resolution*. Entry cùng cuộc thi FSE-AIWare 2026 (chưa công bố; số liệu nhóm tự reproduce trong cùng harness).

Thank you!

Cảm ơn đã lắng nghe!

Questions & Discussion



MEMRES & CGAR: Agentic Python Dependency Resolution

Trần Chí Nguyên · Đào Sỹ Duy Minh · Huỳnh Trung Kiệt

University of Science, VNU-HCM



Backup: MEMRES Stage Details

Stage 1: Intra-Session Memory

- ▶ Cache solution từ snippet đã fixed trong batch
- ▶ Match qua Jaccard ≥ 0.5 trên import sets
- ▶ Reapply qua 1 **pip install** duy nhất
- ▶ Reset giữa runs $\rightarrow$ no cross-run overfitting

Stage 2: Hybrid Evaluation

- ▶ Static regex + **Semantic Import Analyzer**
- ▶ **13 usage patterns**: (import, call) $\rightarrow$ pkg
VD: `Image.open()` $\rightarrow$ Pillow
- ▶ **11 ecosystems** • 11 code patterns
- ▶ **Py2 detector**: 13 Py2 + 5 Py3 indicators

Stage 3: Confidence Cascade (6 levels)

- ▶ L1 Session • L2 Compat Map (40+pkg $\times$ 8Py)
- ▶ L3 Templates (23 sets) • L5 Heuristics (45+)
- ▶ L4 Co-occurrence (50K req.txt, pre-2020)
- ▶ L6 LLM (last resort) • conf $\geq 7 / 3$

Stage 4: Build Loop + Reflexion

- ▶ Docker build, ≤ 10 retries, 180s/build
- ▶ **Sys-dep inject**: 35+ pip $\rightarrow$ apt mappings
- ▶ **Version fallback**: numpy 1.16 $\rightarrow$ 1.15 $\rightarrow$ 1.14
- ▶ **Reflexion** (NeurIPS'23) verbal RL retry

Self-Evolving Memory (*Mobile-Agent-E*)

- ▶ **Tips** = NL guidelines (cross-snippet)
- ▶ **Shortcuts** = validated solutions (Jaccard ≥ 0.5)
- ▶ Sandboxed Docker validate trước khi commit
- ▶ Anti-pattern: per-package failure stats

Error Pattern KB

- ▶ 200+ import $\rightarrow$ pip mappings (15 domains)
- ▶ 35 name corrections • 8 regex rules
- ▶ Version constraints cho 14 packages
- ▶ Self-learning at runtime, no HG2.9K leakage

Hiệu quả: median resolution 15.2s • self-learning at runtime, no HG2.9K leakage

Backup: GitChameleon Re-framing

GitChameleon gốc là **code-generation benchmark** cho LLM “viết code đúng version”. Mình **re-frame** thành **dependency resolution test**.

Gốc: 1 JSON / example

```
{
  "example_id": 0,
  "library": "torch",
  "version": "1.9",
  "python_version": "3.6",
  "starting_code": "import torch\ndef ...",
  "solution": "    import numpy ...\n    ...",
  "test": "assert torch.allclose(...)",
  "extra_dependencies": ["numpy", "scipy"]
}
```

3 phần tách biệt: starting / solution / test
LLM thấy starting + version → generate body → chấm bằng test.

Sau convert: 1 snippet runnable

```
import torch
def log_ndtr(input_tensor):
    import numpy as np
    from scipy.stats import norm
    output = torch.from_numpy(
        norm.logcdf(input_tensor.numpy()))
    return output

# --- test ---
input_tensor = torch.linspace(-10,10,20)
assert torch.allclose(log_ndtr(...), ...)
```

Concat starting+solution+test thành 1 file.
Version ngẫu nhiên → lưu riêng ground_truth.csv.

Tại sao khó hơn HG2.9K:

Success = **pass unit test** (semantic correctness), không phải chỉ import chạy được. Cài torch==2.0 thay vì 1.9 vẫn import OK nhưng assert fail → tool bắt buộc pick đúng version.

Backup: Filter & CSP (I/O)

Filter: CandidateGraphBuilder

candidate_graph_builder.py

```
Input : packages = ["scipy", "numpy"]
       python_version = "3.7"

# Per package:
#   GET https://pypi.org/pypi/<pkg>/json
#   filter requires_python by SpecifierSet
#   wheel filter (linux/x86_64 only):
#     "-none-any.whl" | "manylinux"
#     | "linux_x86_64" | "-cp37-" | "-py3-"
#   sort: wheel-first, newest-first
#   take top 8

Output: {
  "scipy": [
    {v:"1.7.3", requires_python:">=3.7",
     has_wheel:True},
    {v:"1.7.2", ..., has_wheel:True},
    ... up to 8 candidates
  ],
  "numpy": [...]
}
```

CSP: ConstraintSolver

constraint_solver.py:51-123

```
Input : graph (from Filter)
       python_version
       exclude_combo # last failed pick

# 1. viable = filter graph by store
#   (HARD/SOFT/UpperBound pruning)
# 2. greedy: pick newest viable per pkg
# 3. if combo in exclude or infeasible:
#     _next_assignment(): odometer-style
#     enumeration, skip known-bad combos
#     max_attempts = 50 (safety cap)
#     ~~~ logic-only,
#     no Docker calls

Output: {"scipy":"1.7.3","numpy":"1.21.6"}
       or None if exhausted
```

Two Ks: $K_{\text{solve}} = 50 \text{ logical attempts/solve call (dict-lookup, } \mu\text{s)} \cdot K_{\text{build}} \leq 10 \text{ Docker retries/snippet (180s/build, expensive)}.$

Backup: Agent Communication (Message I/O)

cgar_resolver.py • loose-coupled via shared ConstraintStore (no direct calls between agents)

1. Planner → Executor

assignment dict

```
{"scipy": "1.7.3", "numpy": "1.21.6",  
  "matplotlib": "3.5.3"}
```

2. Executor → Analyzer

build result + error log

```
{"success": False,  
  "error_type": "ImportError",  
  "error_log": "ImportError: cannot  
    import name 'imread' from 'scipy.misc'",  
  "assignment": {"scipy": "1.7.3", ...}}
```

3. Analyzer → Store (write)

store.add_upper_bound() / inject()

```
add_upper_bound("scipy", "3.7", "1.2")  
# scipy >= 1.2 infeasible on py3.7  
# (imread removed in scipy 1.2)
```

4. Store → Planner (read)

next round, planner queries store

```
store.get_upper_bound("scipy", "3.7")  
# -> "1.2"  
store.get_infeasible_versions("scipy", "3.7")  
# -> {"1.7.3", "1.6.0", ...}  
# Planner's solver prunes these
```

5. Analyzer → Critic

critic.record_failure() per build

```
{"assignment": {"scipy": "1.7.3", ...},  
  "python_version": "3.7",  
  "error_type": "ImportError"}
```

6. Critic → Orchestrator

strategic action (fires when stuck)

```
{"action": "switch_python",  
  "target": "2.7",  
  "reason": "repeated SyntaxError  
    -> likely Python 2 code",  
  "source": "llm"}
```

Backup: Session Store Data Structures

constraint_store.py • failure_injector.py • shared across all agents in 1 session

1. InfeasibleRecord + 3 stores

```
@dataclass
class InfeasibleRecord:
    package, version, python_version: str
    error_type: ConstraintType # HARD|SOFT
    error_signature: str # normalized log
    confidence: float # 1.0 HARD, 0.8 SOFT
    count: int # ++ on re-observation

    _records: Dict[(pkg,ver,py) -> InfeasibleRecord]
    _combo_records: Dict[(FrozenSet,py) -> sig]
    _upper_bounds: Dict[(pkg,py) -> version_str]
```

2. Regex classifiers

```
_HARD = ["requires a different Python",
         "No matching distribution found", ...]
_SOFT = ["ImportError", "ModuleNotFoundError",
         "cannot import name", ...]
_API_REMOVED_RE =
    r"cannot import name '(\w+)' from '([\w.]+)"
```

Flow: error log → stored record

```
# Input: Docker stderr
log = "ImportError: cannot import name
      'imread' from 'scipy.misc'"

# 1. normalize_error_signature(log)
#    -> strip line-N, 0xADDR, paths

# 2. classify_error(log, assignment)
#    -> (SOFT, sig) # _SOFT match

# 3. _API_REMOVED_RE.search(log)
#    -> ("imread", "scipy.misc")
add_upper_bound("scipy", "3.7", "1.2")

# 4. Analyzer LLM may confirm/add UB
consult_llm() -> {"type": "UPPER",
                 "package": "scipy", "upper_bound": "1.2"}

# Resulting store state:
_records[("scipy", "1.7.3", "3.7")] =
    InfeasibleRecord(SOFT, count=1, 0.8)
_upper_bounds[("scipy", "3.7")] = "1.2"
```

Backup: LLM Call Mechanics

HTTP → Ollama Gemma-2 9B (localhost:11434/api/generate)

1. Request payload

```
{
  "model": "gemma2",
  "prompt": "<agent>",
  "stream": false,
  "format": "json",
  "options": {
    "temperature": 0.3,
    "num_predict": 128
  }
}
```

2. Response (Planner)

```
{
  "assignment": {
    "scipy": "1.1.0",
    "numpy": "1.16.6"
  },
  "reason":
    "imread requires scipy<1.2"
}
```

3. Per-agent budget & trigger

Agent	tok	Khi nào call
Planner	128	mỗi plan step
Analyzer	128	mỗi Docker fail
Critic	96	≥ 3 same-type fails

4. Hard limits

HTTP timeout / call	60 s
Retry on timeout	1×
Docker build timeout	180 s
Max retries / snippet	$K_{\text{build}} \leq 10$
Empty / parse-fail response	→ rule fallback

Backup: Build Loop & Anti-Hallucination

outer orchestration • response validation tại mỗi agent

1. Build loop (per snippet)

reset per-snippet state

loop ≤ 10 Docker attempts:

- ▶ **Planner** $\rightarrow$ PyPI + CSP + LLM
- ▶ **Executor** Docker (180s)
- ▶ pass? on_success(); break
- ▶ else **Analyzer** writes HARD
/SOFT
/UPPER
- ▶ **Critic** fires nếu ≥ 3 same-fails

Store persists qua snippet kể cùng session.

2. Anti-hallucination check

- ▶ **Planner**: version $\in$ candidate graph $\rightarrow$ else solver
- ▶ **Analyzer**: type $\in$ {HARD, SOFT, UPPER} $\rightarrow$ else discard
- ▶ **Critic**: action $\in$ {continue, switch_python, mark_unfixable}
- ▶ **JSON parse fail** $\rightarrow$ regex salvage
- ▶ **Empty / timeout** $\rightarrow$ rule-only path

Telemetry per agent (logged):

llm_consultations $\cdot$ llm_accepted (P) $\cdot$ llm_upper_bounds (A) $\cdot$ llm_overrides (C)

Backup: Planner Agent Prompt

prompts.py • PLANNER_PROMPT • consulted in PlannerAgent.step()

You are the Planner agent in a dependency-resolution system.

Goal: choose one version per package that is most likely to install AND import successfully on Python {python_version}, taking into account constraints learned from previous failures in this session.

Candidate versions per package (newest first, wheel-available marked *):
{candidate_block}

Past failures observed for these packages (do NOT repeat them):
{constraint_block}

Rule-based solver's current pick (your default if you cannot improve it):
{rule_pick}

Rules:

- Pick ONE version per package from the candidates list above.
- Prefer wheel-available (*) versions to avoid source builds.
- Avoid versions appearing in past failures.
- If the rule-based pick looks safe, return it unchanged.

Output ONLY valid JSON: {"assignment": {"pkg1": "X.Y.Z", "pkg2": "X.Y.Z"}, "reason": "<short>"}

JSON output:

Backup: Analyzer Agent Prompt

prompts.py • ANALYZER_PROMPT • consulted after regex classification

You are the Analyzer agent. A Docker build just failed.

Assignment tried:

{assignment_block}

Python version: {python_version}

Regex classifier's initial guess: type={regex_type}, signature={regex_sig}

Error log (truncated):

{error_log}

Rules:

- type in {HARD, SOFT, UPPER}.
 - HARD = deterministic incompat (Python version mismatch, no matching dist).
 - SOFT = runtime error needing >= 2 observations (ImportError, generic build fail).
 - UPPER = API was removed in current version -> infer an upper bound version.
- If you see "cannot import name X from PKG", set type=UPPER and fill upper_bound.
- upper_bound version should be the LAST known-good version before X was removed (your best guess from common-knowledge of the library; null if unsure).

Output ONLY valid JSON:

{"type": "HARD|SOFT|UPPER", "package": "<pkg>", "upper_bound": "X.Y.Z|null", "reason": "<short>"}

JSON output:

Backup: Critic Agent Prompt

prompts.py • CRITIC_PROMPT • fires when ≥ 3 consecutive same-type fails

You are the Critic agent. The Planner has failed several attempts on the current snippet and you must propose a strategic pivot.

Failure pattern summary:

- pattern: {pattern}
- total failures: {total_failures}
- recent error types: {last_error_types}
- python versions already tried: {python_versions_tried}

Available actions (pick exactly one):

- "continue" : keep retrying, no strong signal of structural problem
- "switch_python" : likely Python 2/3 mismatch; try a different Python version
- "mark_unfixable" : structurally infeasible (private package, deprecated API, Py2-only with no Py2 wheel), stop wasting budget

Rules:

- Repeated SyntaxError under Python 3+ and 2.7 not tried -> switch_python target=2.7.
- Repeated ImportError with > 4 failures -> mark_unfixable (likely private/deprecated).
- > 8 attempts and still mixed errors -> mark_unfixable (budget exhausted soon).
- Otherwise -> continue.

Output ONLY valid JSON:

```
{"action": "continue|switch_python|mark_unfixable", "target": "<py_version or null>", "reason": "<short>"}
```

JSON output: